

JomCode Coding Bootcamp

Modules, files and configuration

Understand, predict, type, run, then practice.

What you will be able to do

- 01 Use imports and a main guard to organize a small Python program.
- 02 Read and write UTF-8 text and JSON with pathlib.
- 03 Keep environment configuration outside source files.

Plain meaning → technical view → use

PLAIN MEANING

Split a program into files and save small amounts of data.

TECHNICAL VIEW

Modules and packages organize imports; pathlib handles paths; JSON serializes basic data.

WHEN YOU USE IT

Keep configuration and practice data separate from application logic.

TIP

Run from the chapter folder; keep passwords out of files committed to Git.

Worked example 1: Standard-library namespaces



chapter_10_modules_files_and_configuration.py

```
1 import math
2 from pathlib import Path
3
4 print(math.ceil(4.2))
5 print(Path("backend") / "app" / "main.py")
```

PREDICT What changes when importing Path directly?

Example 1: output and reason

TERMINAL	>_	EXPECTED OUTPUT
----------	----	-----------------

5		
backend/app/main.py	(separator depends on OS)	

WHY THIS HAPPENS

Path is available as a name without a pathlib prefix. Platform-aware joining avoids hardcoded separators.

Exercise 1: Two-file import

BUILD

Create `domain.py` with `remaining_seats(capacity, registered)`. Create `runner.py` that imports the function and prints `remaining_seats(20, 8)`. Run `runner.py` from their shared folder.

CHECK

Direct execution prints 12. Importing `domain` does not prompt, seed or start a server.

IF STUCK

Keep the calculation in `domain.py` and the import/call in `runner.py`.

Worked example 2: JSON round trip



chapter_10_modules_files_and_configuration.py

```
1  import json
2
3  records = [{"title": "Python", "capacity": 20}]
4  text = json.dumps(records, indent=2)
5  restored = json.loads(text)
6  print(restored == records)
7  print(restored[0]["capacity"])
```

PREDICT Does a round trip prove the capacity is valid?

Example 2: output and reason

TERMINAL	>_	EXPECTED OUTPUT
----------	----	-----------------

True		
20		

WHY THIS HAPPENS

It proves serialization consistency, not domain validity.

Exercise 2: File round trip

BUILD

Write a workshop list to a new practice JSON file, read it back and compare. Use an isolated practice directory.

CHECK

UTF-8 title Café survives. Capacity remains integer. File handle closes.

IF STUCK

The instructor test uses TemporaryDirectory so user files remain untouched.

Worked example 3: Explicit configuration



chapter_10_modules_files_and_configuration.py

```
1 import os
2
3 def api_origin():
4     return os.getenv("API_ORIGIN", "http://localhost:8000")
5
6 print(api_origin())
```

PREDICT Is a local URL a safe default database password?

Example 3: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
http://localhost:8000 when API_ORIGIN is absent
```

WHY THIS HAPPENS

No. Safe public defaults and private credentials are different.

Exercise 3: Private database URL

BUILD

Write `require_database_url(values)` that rejects a missing or empty `DATABASE_URL`.

CHECK

Missing and empty values raise `ValueError`.

IF STUCK

Check presence without printing the value.

Checkpoint

PROVE IT

A two-module program saves and reloads JSON using pathlib, and no secret appears in source control.